```
!pip install pandas sentence-transformers Levenshtein jellyfish spaCy scikit-learn jiwer joblib xgboost lightgbm catboost
!python -m spacy download es_core_news_md
```

```
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->ser
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->se
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0-
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0-
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->se
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sente
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->senter
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->ser
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transfor
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->to
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0-
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.:
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2-
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spaCy) (3.0.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (4.62.:
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.5.0
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (3.3.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly->catboost) (9.1.4)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.20.(
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hul
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->hug
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.5.0,>=
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.11.(
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.2(
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.20.0->se
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->hug
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->hu
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0
Downloading levenshtein-0.27.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (153 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 153.3/153.3 kB 8.9 MB/s eta 0:00:00
Downloading jellyfish-1.2.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (360 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 360.5/360.5 kB 30.2 MB/s eta 0:00:00
Downloading jiwer-4.0.0-py3-none-any.whl (23 kB)
Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl (97.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 97.1/97.1 MB 10.4 MB/s eta 0:00:00
Downloading rapidfuzz-3.14.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (3.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 129.4 MB/s eta 0:00:00
Installing collected packages: rapidfuzz, jellyfish, Levenshtein, jiwer, catboost
Successfully installed Levenshtein-0.27.3 catboost-1.2.10 jellyfish-1.2.1 jiwer-4.0.0 rapidfuzz-3.14.3
Collecting es-core-news-md==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/es_core_news_md-3.8.0/es_core_news_md-3.8.0-py3-none
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 42.3/42.3 MB 14.3 MB/s eta 0:00:00
Installing collected packages: es-core-news-md
Successfully installed es-core-news-md-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('es_core_news_md')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
import pandas as pd
import numpy as np
import re
import string
import spacy
import jellyfish
import Levenshtein
import jiwer
import joblib
import random
from sentence_transformers import SentenceTransformer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, classification_report
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.preprocessing import LabelEncoder
from sklearn.utils.class_weight import compute_class_weight

print("Loading NLP Models...")
model = SentenceTransformer('intfloat/multilingual-e5-base')
nlp = spacy.load("es_core_news_md")

random.seed(5)
np.random.seed(5)

# --- 1. NLP & CLEANING FUNCTIONS ---

def clean_transcript(text):
    if pd.isna(text): return ""
    text = str(text)

    # Remove ALL transcriber notes in brackets and parentheses
    text = re.sub(r'\[.*?\]', ' ', text)
    text = re.sub(r'\(.*?\)', ' ', text)

    # Replace hesitation pauses or hyphens with a SPACE
    text = re.sub(r'\.+', ' ', text)
    text = re.sub(r'-+', ' ', text)

    # Remove "xxx" denoting unintelligible audio
    text = re.sub(r'\bx+\b', ' ', text, flags=re.IGNORECASE)

    # Remove English and Spanish punctuation
    spanish_punct = string.punctuation + '¿¡'
    text = text.translate(str.maketrans('', '', spanish_punct))

    # Flatten vowels but PRESERVE the ñ
    replacements = {
        'á': 'a', 'é': 'e', 'í': 'i', 'ó': 'o', 'ú': 'u', 'ü': 'u',
        'Á': 'A', 'É': 'E', 'Í': 'I', 'Ó': 'O', 'Ú': 'U', 'Ü': 'U'
    }
    for accented, flat in replacements.items():
        text = text.replace(accented, flat)

    return " ".join(text.split()).lower()

def check_critical_meaning_loss(target, utterance):
    t_doc = nlp(target)
    u_doc = nlp(utterance)

    t_neg = any(token.text == "no" for token in t_doc)
    u_neg = any(token.text == "no" for token in u_doc)
    if t_neg != u_neg: return True

    antonyms = [{"derecha", "izquierda"}, {"arriba", "abajo"}, {"antes", "despues"}]
    for pair in antonyms:
        word1, word2 = list(pair)
        if (word1 in target and word2 in utterance) or (word2 in target and word1 in utterance):
            return True
    return False

def get_token_phonetic_similarity(target, utterance):
    t_words = target.split()
    u_words = utterance.split()
    if not t_words: return 0.0
    total_score = sum(max([jellyfish.jaro_winkler_similarity(tw, uw) for uw in u_words] + [0.0]) for tw in t_words)
    return total_score / len(t_words)

def get_lexical_similarity(target, utterance):
    if not target or not utterance: return 0.0
    return Levenshtein.ratio(target, utterance)

def get_semantic_similarity(target, utterance):
    if not target or not utterance: return 0.0
    embeddings = model.encode([target, utterance])
    return model.similarity(embeddings[0], embeddings[1]).item()

def get_wer_features(target, utterance):
    """
    Returns 4 distinct metrics instead of a basic 'length penalty'.
    This tells the AI if the student added extra words (Insertions),
    missed words (Deletions), or swapped words (Substitutions).
    """
```

```
    if not target or not utterance:
        return 1.0, 0.0, 1.0, 0.0
    try:
        # Calculate WER alignment
        out = jiwer.process_words(target, utterance)

        # Normalize the metrics based on the target length
        t_len = len(target.split())
        if t_len == 0: t_len = 1

        wer = out.wer
        insertions = out.insertions / t_len
        deletions = out.deletions / t_len
        substitutions = out.substitutions / t_len

        return wer, insertions, deletions, substitutions
    except Exception as e:
        print(f"WER Error on Target: '{target}' | Utterance: '{utterance}' | Error: {e}")
        return 1.0, 0.0, 1.0, 0.0


def generate_synthetic_class_1(text, nlp, drop_prob=0.3, lemma_prob=0.8, swap_prob=0.3):
    """
    Takes perfect Spanish text and degrades it to simulate a Class 1 beginner.
    """
    if not text or str(text) == 'nan': return ""

    doc = nlp(str(text))
    tokens = [token for token in doc]

    if len(tokens) <= 2:
        return text # Too short to meaningfully degrade

    new_tokens = []

    # 1. VERB DE-CONJUGATION (Beginners use root infinitives)
    for token in tokens:
        word = token.text
        if token.pos_ == "VERB" or token.pos_ == "AUX":
            if random.random() < lemma_prob:
                word = token.lemma_ # e.g., converts "habló" to "hablar"
        new_tokens.append(word)

    # 2. RANDOM WORD DROPS (Beginners forget words)
    surviving_tokens = []
    for word in new_tokens:
        # Drop words randomly, but don't delete the whole sentence
        if random.random() < drop_prob and len(surviving_tokens) > 0:
            continue
        surviving_tokens.append(word)

    # 3. SYNTAX SWAPPING (Beginners mess up word order)
    if len(surviving_tokens) > 1 and random.random() < swap_prob:
        idx = random.randint(0, len(surviving_tokens) - 2)
        # Swap two adjacent words
        surviving_tokens[idx], surviving_tokens[idx+1] = surviving_tokens[idx+1], surviving_tokens[idx]

    final_string = " ".join(surviving_tokens)

    # 4. PHONETIC/SPELLING ERRORS (Simulating heavy accents)
    if random.random() < 0.4:
        final_string = final_string.replace('v', 'b')
    if random.random() < 0.2:
        final_string = final_string.replace('h', '') # Drop silent H

    return final_string.strip()

# --- 2. TRAIN THE AI ON HISTORICAL DATA ---

print("\nAggregating historical training data...")
train_file_path = 'Example_EIT Transcription and Scoring Sheet.xlsx'
all_train_sheets = pd.read_excel(train_file_path, sheet_name=None)

data_frames = []
for sheet_name, df in all_train_sheets.items():
    if '_vA' in sheet_name or '_vB' in sheet_name:
        try:
```

```
            temp_df = df[['Stimulus', 'Transcription Rater 1', 'Score Rater 1']].copy()
            temp_df.columns = ['Stimulus', 'Transcription', 'Human_Score']
            data_frames.append(temp_df)
        except KeyError:
            continue

train_df = pd.concat(data_frames, ignore_index=True)
train_df = train_df.dropna(subset=['Human_Score', 'Stimulus', 'Transcription'])
train_df['Human_Score'] = pd.to_numeric(train_df['Human_Score'], errors='coerce')
train_df = train_df.dropna(subset=['Human_Score']).astype({'Human_Score': 'int'})

print("Generating Synthetic Class 1 (Beginner) Data...")

# Isolate the perfect Class 4 responses
class_4_data = train_df[train_df['Human_Score'] == 4].copy()

# We will generate 3 synthetic beginner sentences for every 1 perfect sentence
synthetic_rows = []
for _ in range(3):
    synth_batch = class_4_data.copy()

    # Degrade the transcription
    synth_batch['Transcription'] = synth_batch['Stimulus'].apply(
        lambda x: generate_synthetic_class_1(x, nlp)
    )

    # Force the label to be a 1
    synth_batch['Human_Score'] = 1
    synthetic_rows.append(synth_batch)

# Combine the synthetic rows into a DataFrame
synthetic_df = pd.concat(synthetic_rows, ignore_index=True)

# Append the synthetic beginners back into the main training data
train_df = pd.concat([train_df, synthetic_df], ignore_index=True)

# Shuffle the dataset so the synthetic data is mixed in nicely
train_df = train_df.sample(frac=1, random_state=42).reset_index(drop=True)

print(f"Added {len(synthetic_df)} synthetic Class 1 examples!")

print("Extracting NLP Features for AI Training...")
train_df['Cleaned_Target'] = train_df['Stimulus'].apply(clean_transcript)
train_df['Cleaned_Utterance'] = train_df['Transcription'].apply(clean_transcript)

train_df['Lexical'] = train_df.apply(lambda row: get_lexical_similarity(row['Cleaned_Target'], row['Cleaned_Utterance']), axis=1
train_df['Semantic'] = train_df.apply(lambda row: get_semantic_similarity(row['Cleaned_Target'], row['Cleaned_Utterance']), axis=
train_df['Phonetic'] = train_df.apply(lambda row: get_token_phonetic_similarity(row['Cleaned_Target'], row['Cleaned_Utterance']),
train_df['Antonym_Swap'] = train_df.apply(lambda row: int(check_critical_meaning_loss(row['Cleaned_Target'], row['Cleaned_Utterar
train_df[['WER', 'Insertions', 'Deletions', 'Substitutions']] = train_df.apply(
    lambda row: pd.Series(get_wer_features(row['Cleaned_Target'], row['Cleaned_Utterance'])), axis=1
)

X = train_df[['Lexical', 'Semantic', 'Phonetic', 'Antonym_Swap', 'WER', 'Insertions', 'Deletions', 'Substitutions']]
y = train_df['Human_Score']

# --- DATA SPLITTING ---

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- ENCODING TARGET ---

print("Encoding Target Variables...")
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

# --- STACKED MODEL ---

print("Initializing Base Models...")
xgb_model = XGBClassifier(random_state=42, eval_metric='mlogloss', use_label_encoder=False)
lgbm_model = LGBMClassifier(random_state=42, class_weight='balanced', verbose=-1)
cat_model = CatBoostClassifier(random_state=42, verbose=0,auto_class_weights='Balanced')

print("Building Stacking Ensemble & Meta-Classifier...")
meta_model = LogisticRegression(max_iter=1000)

stacked_model = StackingClassifier(
```

```
stacked_model = StackingClassifier(
    estimators=[
        ('xgb', xgb_model),
        ('lgbm', lgbm_model),
        ('cat', cat_model)
    ],
    final_estimator=meta_model,
    cv=5,
    n_jobs=-1
)

print("Training Stacking Classifier with Manual Weights...")
stacked_model.fit(X_train, y_train_enc)

print("ML Engine Trained Successfully!")

print("\nApplying Custom Probability Thresholds...")
# Get the raw percentages for every class instead of the final guess
y_pred_probs = stacked_model.predict(X_test)

# Calculate new metrics
accuracy = accuracy_score(y_test_enc, y_pred_probs)
print(f"Threshold-Tuned Accuracy: {accuracy * 100:.2f}%\n")

print(classification_report(y_test, y_pred_probs, zero_division=0))

# --- 3. SCORE THE TEST FILES ---

print("\nScoring the final Evaluation Test Participants...")
test_file_path = 'AutoEIT Sample Transcriptions for Scoring.xlsx'
all_test_sheets = pd.read_excel(test_file_path, sheet_name=None)

# Export the final predictions to a new Excel file
with pd.ExcelWriter('AutoEIT_Developer_Matrix_Full_Data.xlsx') as writer:
    for sheet_name, test_df in all_test_sheets.items():
        if sheet_name == 'Info':
            continue

        print(f" -> Grading Participant: {sheet_name}")

        # Clean Test Data
        test_df['Cleaned_Target'] = test_df['Stimulus'].apply(clean_transcript)
        test_df['Cleaned_Utterance'] = test_df['Transcription Rater 1'].apply(clean_transcript)

        # Extract features for Test Data
        test_df['Lexical'] = test_df.apply(lambda row: get_lexical_similarity(row['Cleaned_Target'], row['Cleaned_Utterance']), a
        test_df['Semantic'] = test_df.apply(lambda row: get_semantic_similarity(row['Cleaned_Target'], row['Cleaned_Utterance']),
        test_df['Phonetic'] = test_df.apply(lambda row: get_token_phonetic_similarity(row['Cleaned_Target'], row['Cleaned_Utterar
        test_df['Antonym_Swap'] = test_df.apply(lambda row: int(check_critical_meaning_loss(row['Cleaned_Target'], row['Cleaned_U
        test_df[['WER', 'Insertions', 'Deletions', 'Substitutions']] = test_df.apply(
    lambda row: pd.Series(get_wer_features(row['Cleaned_Target'], row['Cleaned_Utterance'])), axis=1
)

        # Predict the scores using our AI
        X_test = test_df[['Lexical', 'Semantic', 'Phonetic', 'Antonym_Swap', 'WER', 'Insertions', 'Deletions', 'Substitutions']]
        test_df['Predicted_Score'] = stacked_model.predict(X_test)

        # Save to the new Excel file
        test_df.to_excel(writer, sheet_name=sheet_name, index=False)

print("\n All done! 'AutoEIT_Developer_Matrix_Full_Data.xlsx' ")
joblib.dump(stacked_model, 'autoeit_model.pkl')
```

```
Loading NLP Models...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100%                                                  387/387 [00:00<00:00, 44.3kB/s]

README.md:       179k/? [00:00<00:00, 12.8MB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
sentence_bert_config.json: 100%                                     57.0/57.0 [00:00<00:00, 6.73kB/s]

config.json: 100%                                                   694/694 [00:00<00:00, 76.7kB/s]

model.safetensors: 100%                                             1.11G/1.11G [00:08<00:00, 146MB/s]

Loading weights: 100%                                               199/199 [00:00<00:00, 563.95it/s, Materializing param=pooler.dense.weight]
XLMRobertaModel LOAD REPORT from: intfloat/multilingual-e5-base
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%                                         418/418 [00:00<00:00, 21.7kB/s]

sentencepiece.bpe.model: 100%                                       5.07M/5.07M [00:00<00:00, 22.2MB/s]

tokenizer.json: 100%                                                17.1M/17.1M [00:00<00:00, 71.5MB/s]

special_tokens_map.json: 100%                                       280/280 [00:00<00:00, 28.5kB/s]

config.json: 100%                                                   200/200 [00:00<00:00, 22.0kB/s]


Aggregating historical training data...
Generating Synthetic Class 1 (Beginner) Data...
Added 2859 synthetic Class 1 examples!
Extracting NLP Features for AI Training...
Encoding Target Variables...
Initializing Base Models...
Building Stacking Ensemble & Meta-Classifier...
Training Stacking Classifier with Manual Weights...
ML Engine Trained Successfully!

Applying Custom Probability Thresholds...
Threshold-Tuned Accuracy: 93.44%

              precision    recall  f1-score   support

           0       1.00      0.81      0.89        21
           1       0.95      0.99      0.97       570
           2       0.56      0.16      0.25        31
           3       0.72      0.83      0.77        52
           4       0.98      0.95      0.96       210

    accuracy                           0.93       884
   macro avg       0.84      0.75      0.77       884
weighted avg       0.93      0.93      0.93       884


Scoring the final Evaluation Test Participants...
 -> Grading Participant: 38001-1A
 -> Grading Participant: 38002-2A
 -> Grading Participant: 38004-2A
 -> Grading Participant: 38006-2A

 All done! 'AutoEIT_Developer_Matrix_Full_Data.xlsx'
['autoeit_model.pkl']
```